



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Sesión 3: Funciones como Ciudadanos de Primera Clase

Semillero en Ciencias de la Computación

Joel Ostos Juan Pedrozo

Mayo 15, 2026



1. Recapitulando
2. Funciones de primera clase
 - ¿Que es una función de orden superior?
 - Funciones como datos
 - Funciones de Primer orden vs Orden Superior
 - Importancia de las funciones de Orden Superior.
 - Importancia de las funciones de Orden Superior.
 - Importancia de las funciones de Orden Superior.
3. Aplicación en Clojure
 - Entendiendo las funciones anónimas
 - Funciones de Orden Superior: Nuestras principales herramientas
 - Map
 - Definición en Clojure
 - Evaluación en Clojure
 - Complejidad del Map
 - Un pequeño vistazo bajo el mantel.
 - Intuición e Idea Mental de Map
 - Reduce

1. Recapitulando

2. Funciones de primera clase

¿Que es una función de orden superior?

Funciones como datos

Funciones de Primer orden vs Orden Superior

Importancia de las funciones de Orden Superior.

Importancia de las funciones de Orden Superior.

Importancia de las funciones de Orden Superior.

3. Aplicación en Clojure

Entendiendo las funciones anónimas

Funciones de Orden Superior: Nuestras principales herramientas

Map

Definición en Clojure

Evaluación en Clojure

Complejidad del Map

Un pequeño vistazo bajo el mantel.

Intuición e Idea Mental de Map

Reduce



- **Funciones Puras:** Retornan un solo valor que depende únicamente de sus argumentos, sin mutar ningún valor existente (cero efectos secundarios).
- **Inmutabilidad:** Una vez que un dato es creado, no puede ser cambiado. En lugar de modificarlo, las funciones puras hacen copias de los datos.
- **Programación Funcional:** Se puede definir formalmente como la programación usando funciones puras que manipulan valores inmutables.



- Modificar una estructura original en memoria rompe nuestra intuición y es una de las mayores causas de errores (bugs).
- **Estado mutable compartido:** Ocurre cuando un estado puede ser modificado y además accedido por diferentes partes del código al mismo tiempo.
- En lenguajes como Java, tener múltiples funciones apuntando a la misma referencia mutable produce resultados incoherentes y viola el principio de transparencia referencial.

- **Inmutabilidad por defecto:** A diferencia de Java, en Clojure todas las colecciones base (vectores, listas y mapas) son inmutables desde su diseño.
- **Copias Eficientes:** Se utiliza *Structural Sharing* bajo el capó para que crear copias nuevas con los cambios deseados no sea costoso en memoria.
- Funciones de actualización como `conj` y `assoc` siempre retornan un nuevo valor, manteniendo la pureza estructural y el original intacto.



- **Función Pura (IVA):** Creación de una función `calcular-iva` que recibe un precio y le suma el 19 %, garantizando que a misma entrada devuelve la misma salida, sin imprimir ni mutar nada.
- **Estado Inmutable (Usuario):** Se definió un sistema de puntos usando un mapa, empleando `assoc` en la función `agregar-puntos` para retornar un nuevo estado, asegurando que el usuario original siga intacto.

1. Recapitulando

2. Funciones de primera clase

¿Que es una función de orden superior?

Funciones como datos

Funciones de Primer orden vs Orden Superior

Importancia de las funciones de Orden Superior.

Importancia de las funciones de Orden Superior.

Importancia de las funciones de Orden Superior.

3. Aplicación en Clojure

Entendiendo las funciones anónimas

Funciones de Orden Superior: Nuestras principales herramientas

Map

Definición en Clojure

Evaluación en Clojure

Complejidad del Map

Un pequeño vistazo bajo el mantel.

Intuición e Idea Mental de Map

Reduce

¿Qué es una función de orden superior?



- Recibe funciones como parametros o devuelve una función.

$$\lambda f. \lambda x. fx$$

```
(defn aplicar [f x]  
  (f x))
```

```
(defn sumar [x]  
  (fn [y]  
    (+ x y)))
```

Las funciones se tratan como datos pues al igual que los datos, se pueden:

- Pasar.
- Guardar.
- Retornar.

Funciones de Primer orden vs Orden Superior

- Las funciones de primer orden no reciben funciones como parametros.
- No devuelven funciones.
- Solo trabaja con tipos "simples".

Importancia de las funciones de Orden Superior.



¿El siguiente código es corresponde a una función pura?

```
static List<String> rankedWords( List<String> words ) {  
    return words.stream()  
        .sorted(scoreComparator)  
        .collect(Collectors.toList());  
}
```



¿El siguiente código es corresponde a una función pura?

```
static List<String> rankedWords( List<String> words ) {  
    return words.stream()  
        .sorted(scoreComparator)  
        .collect(Collectors.toList());  
}
```

No, pues, tenemos una función **scoreComparator** que no se define dentro del scope de nuestra misma función, es decir, la *signature* de nuestra función no nos dice la información completa de lo que hace, y una función pura debe hacerlo.



¿El siguiente código es corresponde a una función pura?

```
static List<String> rankedWords( Comparator<String> comp ,  
    List<String> words ) {  
    return words.stream()  
        .sorted(comp)  
        .collect(Collectors.toList());  
}
```

Claramente, pues ahora la firma de la función corresponde con lo que hace, además, nos da total libertad de modificar el comparador, y además, esta es una función de Orden Superior pues toma como parametro otra función (comp).

1. Recapitulando
2. Funciones de primera clase
 - ¿Que es una función de orden superior?
 - Funciones como datos
 - Funciones de Primer orden vs Orden Superior
 - Importancia de las funciones de Orden Superior.
 - Importancia de las funciones de Orden Superior.
 - Importancia de las funciones de Orden Superior.
3. Aplicación en Clojure
 - Entendiendo las funciones anónimas
 - Funciones de Orden Superior: Nuestras principales herramientas
 - Map
 - Definición en Clojure
 - Evaluación en Clojure
 - Complejidad del Map
 - Un pequeño vistazo bajo el mantel.
 - Intuición e Idea Mental de Map
 - Reduce

Aplicación en Clojure

¿Qué es una Función Anónima?



- Una función anónima es, simplemente, una función que **no tiene nombre**.
- Se crean sobre la marcha (on the fly) para ser usadas en el momento.
- **El equivalente en otros lenguajes:** Esto es exactamente lo que en lenguajes como Python, Java, C++ o Ruby se conoce como **Funciones Lambda** (o *Arrow Functions* en JavaScript).
- Son extremadamente útiles cuando necesitamos pasar una operación rápida a funciones de orden superior como `map`, `filter` o `reduce`.

- La forma más explícita de crear una lambda en Clojure es usando la palabra reservada `fn`.
- **Estructura:** `(fn [argumentos] cuerpo)`

Ejemplo: Una función que recibe un número y lo multiplica por 2.

Código Clojure

```
(fn [x] (* x 2))
```

Si la usamos inmediatamente:

Evaluación

```
((fn [x] (* x 2)) 5)  
;; Devuelve: 10
```

- Como los programadores en Clojure usan lambdas todo el tiempo, existe una sintaxis reducida (un *reader macro*) que nos ahorra escribir `fn` y los corchetes.
- **Estructura:** `#{cuerpo-de-la-funcion%}`
- Los argumentos se representan con el símbolo de porcentaje %.

Reglas de los argumentos:

- % o %1: Representa el primer argumento.
- %2: Representa el segundo argumento.
- %3, %4, etc.: Argumentos sucesivos.

Veamos cómo la misma "lambda" se va reduciendo con el atajo:

Ejemplo 1: Sumar 10 a un número

- Con fn: `(fn [x] (+ x 10))`
- Con atajo: `#+% 10`

Ejemplo 2: Sumar dos números

- Con fn: `(fn [x y] (+ x y))`
- Con atajo: `#+%1 %2`

¿Para qué las usamos en la vida real?



Su mayor poder está al combinarlas con Funciones de Orden Superior (colecciones).

Multiplicar por 10 todos los elementos de un vector:

map con fn

```
(map (fn [x] (* x 10)) [1 2 3 4])  
;; => (10 20 30 40)
```

map con sintaxis corta (Lambda / Atajo)

```
(map #(* % 10) [1 2 3 4])  
;; => (10 20 30 40)
```

- **Funciones Anónimas == Lambdas:** Funciones sin nombre para uso rápido.
- `fn [args] ...`: Sintaxis completa, ideal para lambdas complejas, con múltiples expresiones o cuando necesitamos usar destructuración.
- `#(... % ...)`: Sintaxis azucarada, ideal para operaciones matemáticas o transformaciones de una sola línea de código.

¡El dominio de las lambdas es el corazón de la programación funcional!

- Map.
- Reduce.
- Filter.

¿Qué es el Map?



- Función de Orden Superior.
- Aplica una función a cada elemento.
- Devuelve una **nueva colección**.

¿Qué es el Map?



```
(map f coll)
```

Dónde f es una función que se toma como parametro y $coll$ es una colección (lista, vector, set, etc...).

El resultado es una nueva colección que es el producto de aplicar f a cada elemento de $coll$

Map usa evaluación perezosa, o lo que es lo mismo, cada operación es calculada si y solo si se usará finalmente, por lo tanto no tiene ningún problema de rendimiento en las colecciones infinitas" que provee Clojure. Hace uso de las **lazy sequences**

¿Qué es una secuencia?

Es una abstracción sobre datos ordenados. Permite recorrer elementos uno a uno.

Una lazy sequence es una secuencia en la que cada objeto se produce bajo demanda

- **Tiempo:** $O(n)$ pues es la aplicación de una función a cada elemento de la colección, por tanto debe recorrerse completamente la colección.

$$\{f_{x_1}, f_{x_2}, \dots, f_{x_n}\}$$

- **Memoria:** $O(1)$ (incremental) ¿Por qué?

¿Por qué es tan eficiente en memoria?

Como se mencionó antes, Map es una función lazy, es decir, no tiene por qué calcular todo el resultado ni copiar los elementos ni guardarlos en ninguna parte, sino que hace función de los **streams** para que según se necesite Map de un resultado, es decir.

Map depende del contexto que la acompaña.

Map no hace esto:

$$[Input\ completo] \rightarrow [Output\ Completo]$$

- No es un loop tradicional.
- Es una transformación declarativa.



- Reutilizable.
- Componible.
- Junto con Filter y Reduce son el **core** de las transformaciones en la programación funcional.

¿Qué es Reduce?



- Función de Orden Superior.
- Combina todos los elementos en uno solo.

¿Qué es Reduce?

```
(reduce f coll)  
(reduce f init coll)
```

Dónde f es una función acumuladora de dos argumentos que se toma como parametro y $coll$ es una colección (lista, vector, set, etc...).

El resultado es una nueva colección que es el producto de aplicar f a los primeros dos elementos de $coll$, luego usar ese resultado y aplicarle f a dicho resultado y al tercer elemento de $coll$, etc...



- Formalmente se le suele llamar fold.
- Es la base de muchas operaciones funcionales.

Muchas funciones se pueden expresar como reduce:

- sum
- map
- filter

A pesar de todo, Reduce no es **lazy** por su definición no es difícil intuirlo pues para acumular todo en un solo elemento solo es posible hacerlo si se aplica la transformación a todos los elementos.

- **Tiempo:** $O(1)$.
- **Memoria:** $O(1)$. Pues solo lleva el acumulador.

¿Cómo funciona por debajo?

Iterativamente podemos verla como la aplicación de la función que se pasó como parametro.

$$(f(f(finitx_1)x_2)x_3)...$$

Toma el acumulador, aplica f al siguiente elemento y repite hasta llegar al final de la secuencia.

La implementación podría verse así:

```
(loop [acc init
      xs coll]
  (if (empty? xs)
      acc
      (recur (f acc (first xs))
              (rest xs)))))
```

En la recursividad tradicional, el modelo se rige por una ejecución en diferido: primero se lanzan las llamadas recursivas y, solo tras alcanzar el caso base, se procede a recuperar los valores de retorno para calcular el resultado final. En este esquema, el sistema no obtiene el fruto del cálculo hasta que se ha completado el retroceso (unwinding) de cada llamada en la pila; es decir, cada paso queda en suspenso, ocupando memoria, a la espera de lo que devuelva el siguiente nivel.

```
public int factorialTradicional(int n) {  
    if (n <= 1) {  
        return 1;  
    }  
    // El calculo (n *) ocurre DESPUES de la llamada  
    // recursiva  
    return n * factorialTradicional(n - 1);  
}
```

Por el contrario, en la recursividad de cola, el cálculo es prioritario y precede a la llamada. El proceso opera de forma acumulativa: se computa el estado actual y se transfiere el resultado directamente al siguiente paso recursivo como un argumento. Esto permite que la instrucción final sea estrictamente la llamada a la función: (return (funcion-recursiva parametros)).

```
public int factorialTail(int n, int acumulador) {  
    if (n <= 1) {  
        return acumulador;  
    }  
    // El calculo (n * acumulador) ocurre ANTES de la  
    llamada recursiva  
    return factorialTail(n - 1, n * acumulador);  
}
```

Esencialmente, la recursividad de cola transforma el flujo en algo similar a un bucle; el valor de retorno de un paso es, sin alteraciones, el valor del siguiente, lo que permite al compilador optimizar el espacio y evitar el desbordamiento de la pila (stack overflow), ya que no es necesario mantener el contexto de los pasos anteriores.

¿Qué es Filter?

- Función de Orden Superior.
- Selecciona elementos que cumplan una condición.

¿Qué es Filter?



```
(filter pred coll)
```

Dónde *pred* es una función booleana (retorna falso/verdadero) que se toma como parametro y *coll* es una colección (lista, vector, set, etc...).

El resultado son los elementos que cumplen el predicado.



- Sencillamente basada en lógica booleana.

A diferencia que Reduce y al igual que Map, Filter es de ejecución **lazy**, usa **lazy sequences**.



- **Tiempo:** $O(n)$.
- **Memoria:** $O(1)$ incremental.

¿Cómo funciona por debajo?



Es sencillamente una pregunta, por cada elemento que sea requerido se preguntará si cumple la condición, en el caso de ser afirmativo entonces se incluye en la "bolsa" que llevamos y en el caso opuesto es descartado y se continúa con la ejecución.

```
(lazy-seq
  (when-let [s (seq coll)]
    (let [x (first s)]
      (if (pred x)
        (cons x (filter pred (rest s)))
        (filter pred (rest s)))))))
```

Map vs Reduce vs Filter



Característica	map	filter	reduce
Operación	Transformar	Filtrar	Colapsar
Entrada	Colección	Colección	Colección (+ init)
Salida	Colección	Subconjunto	Valor
Lazy	Sí	Sí	No
Tiempo	$O(n)$	$O(n)$	$O(n)$
Espacio	$O(1)$ lazy	$O(1)$ lazy	$O(1)$
Cómo	Aplica función	Evalúa predicado	Función acumuladora
Construcción	No modifica	No modifica	Colapsa
Uso	Transformar datos	Filtrar datos	Reducir a valor

Cuadro: Comparación de funciones de orden superior en Clojure



1. Recapitulando
2. Funciones de primera clase
 - ¿Que es una función de orden superior?
 - Funciones como datos
 - Funciones de Primer orden vs Orden Superior
 - Importancia de las funciones de Orden Superior.
 - Importancia de las funciones de Orden Superior.
 - Importancia de las funciones de Orden Superior.
3. Aplicación en Clojure
 - Entendiendo las funciones anónimas
 - Funciones de Orden Superior: Nuestras principales herramientas
 - Map
 - Definición en Clojure
 - Evaluación en Clojure
 - Complejidad del Map
 - Un pequeño vistazo bajo el mantel.
 - Intuición e Idea Mental de Map
 - Reduce

Ejercicios Prácticos

Ejercicio 1: Usando Map



Objetivo: Dado un vector de números [1 2 3 4 5], usar map para obtener un nuevo vector con el cuadrado de cada número.

Ejercicio 2: Usando Reduce



Objetivo: Usar reduce para calcular la multiplicación total de los elementos del vector.

Ejercicio 3: Usando Filter



Objetivo: Usar filter para obtener solo los números pares del vector anterior

Ejercicio 4: Usando los 3



Objetivo: Combinar las 3 funciones presentadas para calcular la suma de los cuadrados de los números impares de una lista.

Gracias!